

Text as Data

Part 2

Meta lets Llama 3 LLM out to graze, claims it can give Google and Anthropic a kicking

Plans multilingual 400 billion parameter version, but starts with more modest fare

 [Tobias Mann](#)

Fri 19 Apr 2024 // 00:57 UTC

Meta has unleashed its latest large language model (LLM) – named Llama 3 – and claims it will challenge much larger models from the likes of Google, Mistral, and Anthropic.

Revealed in a lengthy [announcement](#) on Thursday, Llama 3 is available in versions ranging from eight billion to over 400 billion parameters. For reference, OpenAI and Google's largest models are nearing two trillion parameters.

For now, we're only getting access to Llama 3's eight billion and 70 billion parameter text variants. Meta isn't done training its largest and most complex models just yet, but hints they will be multilingual and multimodal – meaning they're assembled from multiple smaller domain-optimized models.

DeepSeek. Temu. TikTok. China Tech Is Starting to Pull Ahead.

May 5, 2025



Big Picture

- Text data are traces of people
- Supervised versus unsupervised:
Do **we tell the computer** what we want, or let the **computer tell us**?
- Methods for extraction
 - Count words with dictionary
 - Impose “Parts of speech”
 - Count named entities

Today: Unsupervised Analysis

1. Bag of words representation of text processing
2. Topic modeling
3. Implementation in Python

Understanding AirBnB Data

Objective: Understand 1) words people use to describe AirBnb listing and 2) neighborhood change (gentrification?)

Text mining approach: Build dictionary of words that signal gentrification

Cute, Safe, pour over

Drawback: You don't know words in advance and you might miss things

Understanding AirBnB Data

Objective: Understand 1) words people use to describe AirBnb listing and 2) neighborhood change (gentrification?)

Text mining approach: Build dictionary of words that signal gentrification

Cute, Safe, pour over

Drawback: You don't know words in advance and you might miss things

Unsupervised methods attempt to use all the words in each document!

Tokenization and Bag-of-Words Model

- **Token:** Unit of analysis and usually a word
- Each document is a “bag of words”: Order does not matter

```
['clean', '&', 'quiet', 'apt', 'home', 'by', 'the', 'park']  
['skylit', 'midtown', 'castle']  
['the', 'village', 'of', 'harlem', '....', 'new', 'york', '!']  
['cozy', 'entire', 'floor', 'of', 'brownstone']
```

- There is a lot of useless information

Object

Bag of 'words'



Tokenization and Bag-of-Words Model

- **Token:** Unit of analysis and usually a word
- Each document is a “bag of words”: Order does not matter

```
['clean', '&', 'quiet', 'apt', 'home', 'by', 'the', 'park']
```

```
['skylit', 'midtown', 'castle']
```

```
['the', 'village', 'of', 'harlem', '....', 'new', 'york', '!']
```

```
['cozy', 'entire', 'floor', 'of', 'brownstone']
```

Step 1: Get rid of stuff (stopwords) you don't want

Why this so early? Text processing can be computationally heavy!

Get rid of words you don't need as soon as possible.

```
## call the specific module
from nltk.corpus import stopwords

## call a specific set of stopwords from package
list_stopwords = stopwords.words('english')

## augment with your own
list_stopwords = stopwords.words("english")

custom_words_toadd = ['apartment', 'new york', 'nyc',
                      'bronx', 'brooklyn',
                      'manhattan', 'queens',
                      'staten island']

list_stopwords_new = list_stopwords + custom_words_toadd
```

Step 2: Lowercase and remove stopwords

```
## convert to lowercase and a list
corpus_lower = ab.name.str.lower().to_list()

## use wordpunct tokenize and filter out with one
example_listing = corpus_lower[0]
nostop_listing = [word for word in wordpunct_tokenize(
    example_listing)
                  if word not in list_stopwords_new])
nostop_listing
```

Before: ['cozy', 'entire', 'floor', 'of', 'brownstone']

After: ['cozy', 'entire', 'floor', 'brownstone']

What is a stem?

CONNECT

CONNECTED

CONNECTING

CONNECTION

CONNECTIONS

If Gensim is failing....

In terminal:

```
pip uninstall gensim
```

```
conda install gensim
```

Step 3: Stem and additional processing

Initialize Stemmer

Run stemmer on token

```
## initialize stemmer
porter = PorterStemmer()

## apply to one tokenized text by iterating
## over the tokens in the text
example_listing_preprocess = [porter.stem(token)
                               for token in nostop_listing
                               if token.isalpha() and
                               len(token) > 2]
```

Output:

```
['cozi', 'entir', 'floor', 'brownston']
```

Only words and greater
than two letters

Pre-processing Ideology



Your turn

Apply **preprocessing** to all listings without relying on a nested for loop.

1. Create a function that applies all preprocessing steps.
2. Iterate over all listings

For every listing, output is tokenized and stemmed words

https://github.com/herbertfreeze/QSS20-X26/blob/main/activities/05_textasdata_partII_topicmodeling_blank.ipynb

BOW Step 1: Collect document

It was the best of times

it was the worst of times

it was the age of wisdom

it was the age of foolishness

How many unique words are there?

BOW Step 1: Collect document

It was the best of times

it was the worst of times

it was the age of wisdom

it was the age of foolishness

- “it”
- “was”
- “the”
- “best”
- “of”
- “times”
- “worst”
- “age”
- “wisdom”
- “foolishness”

BOW Step 1: Collect document

It was the best of times

- “it” = 1
- “was” = 1
- “the” = 1
- **“best” = 1**
- “of” = 1
- “times” = 1
- “worst” = 0
- “age” = 0
- “wisdom” = 0
- “foolishness” = 0

BOW Step 1: Collect document

it was the worst of times

- “it” = 1
- “was” = 1
- “the” = 1
- “best” = 0
- “of” = 1
- “times” = 1
- **“worst” = 1**
- “age” = 0
- “wisdom” = 0
- “foolishness” = 0

BOW Step 1: Collect document

it was the age of wisdom

- “it” = 1
- “was” = 1
- “the” = 1
- “best” = 0
- “of” = 1
- “times” = 0
- “worst” = 0
- **“age” = 1**
- **“wisdom” = 1**
- “foolishness” = 0

BOW Step 1: Collect document

it was the age of foolishness

- “it” = 1
- “was” = 1
- “the” = 1
- “best” = 0
- “of” = 1
- “times” = 0
- “worst” = 0
- “age” = 1
- “wisdom” = 0
- **“foolishness” = 1**

Document-Word Matrix | How computers see words

"it was the best of times"	=	[1, 1, 1, 1, 1, 1, 0, 0, 0, 0]
"it was the worst of times"	=	[1, 1, 1, 0, 1, 1, 1, 0, 0, 0]
"it was the age of wisdom"	=	[1, 1, 1, 0, 1, 0, 0, 1, 1, 0]
"it was the age of foolishness"	=	[1, 1, 1, 0, 1, 0, 0, 1, 0, 1]

Sparse versus dense matrix

s p a r s e

	7					6
	7	6	3		4	
	4	3				
4	2					
				3	2	4

© Matt Eding

DENSE

0	7	0	0	0	0	6
0	7	6	3	0	4	0
0	4	3	0	0	0	0
4	2	0	0	0	0	0
0	0	0	0	3	2	4

How would you do this in Python?

How would you do this in Python?

1. Find all words with a loop
2. Make an empty dataframe with words as the headers
3. Go through all the listings again, count the number of times each token occurs, then put corresponding number into the DTM

Skip all that and use **sklearn**

Document-Term Matrix (DTM)

Words take up a lot of memory. DTMs recode words into a matrix.

- Each row is a document (AirBnb listing)
- Each column is a token
- Values indicate how many times a term occurs

doc	1br	apartment	apt	area	backyard	bdrm ...
1	0	0	1	0	0	0
2	0	0	0	2	0	0
3	0	0	0	0	0	0
4	0	0	0	0	3	0
5	0	0	1	0	0	0

```
def create_dtm(list_of_strings, metadata):
    ## init tokenizer and apply to list of documents
    vectorizer = CountVectorizer(lowercase = True)
    dtm_sparse = vectorizer.fit_transform(list_of_strings)
    ## convert to (1) dense mat; (2) dataframe and (3) add metadata
    dtm_dense_named = pd.DataFrame(dtm_sparse.todense(),
                                    columns=vectorizer.get_feature_names())
    dtm_dense_named_withid = pd.concat([metadata.reset_index(),
                                        dtm_dense_named], axis = 1)
    return (dtm_dense_named_withid)
```

CountVectorizer: Makes a sklearn tokenizer to preprocess the string

vectorizer.fit_transform() : Feed in list of documents and it outputs a list of tuples.

The tuples are (term, document)

dtm_sparse.to_dense() : Change to dense format

Last part: Concatenates the missing columns (metadata). Don't worry about this!

Your turn (Part 2)

What is topic modeling?

We have a bunch of text documents, we want to know the key topics.

Machine learning models have “internal representations” of the **semantic world**

Twitter fights about AI world views:

- Original post: <https://twitter.com/wesg52/status/1709551516577902782>
- Clap back: <https://twitter.com/acidflask/status/1709914015919403320>

Topic Modeling | Latent Dirichlet Allocation (LDA)

- We have n topics
- Each document is a mixture over all topics.
- Each topic is a mixture over words

LDA Estimates...

Distribution of words over each topic and
Proportion of documents in each topic.

Say you have
a lot of people.



Each writes
some texts



That discuss a few
different topics

T1: Politics

congress, nations,
power, votes, agree-
ment, bargaining

T2: Statistics

estimator, data, ana-
lysis, variance, model,
inference

The Latent Dirichlet Allocation estimates:

- 1 The topics- each is a distribution over words
- 2 The proportion of each document in each topic

How to implement this? Co-occurrence

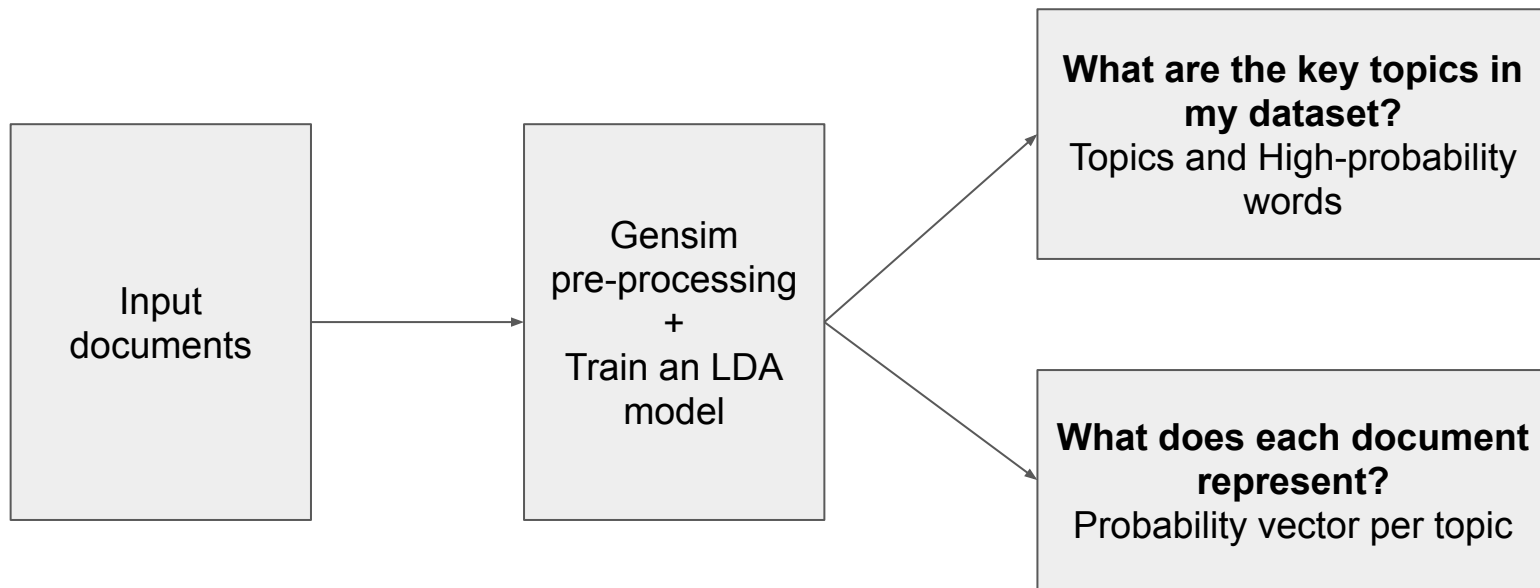
	Word ₁	Word ₂	...	Word _J
Doc ₁	0	1	...	0
Doc ₂	2	0	...	3
⋮	⋮	⋮	⋮	⋮
Doc _N	0	1	...	1

We are learning how words occur together.

The model wants a **topic to contain as few words** as possible. And document wants **as few topics as possible**. Less is more.

Python implementation: Gensim

Gensim is a package that does all of this complicated procedures for you.



Specific Procedures

Create a dictionary of words.

The is the universe of allowed tokens in your model

Filter out words you don't want.

Some words happen too much (COVID-19) or too little.

I.e. If you have 1 million documents, no need for words < 1 time

Create a corpus of words by apply the dictionary

Run LDA

Prep your data to load into Gensim LDA

```
## Step 1: tokenize documents and store in list
text_raw_tokens = [wordpunct_tokenize(one_text)
                    for one_text in ab_small.name_lower]
## Step 2: use gensim create dictionary – gets all unique
            words across documents
text_raw_dict = corpora.Dictionary(text_raw_tokens)
## Step 3: filter out very rare and very common words
            from dictionary; feeding it n docs as lower and upper
            bounds
text_raw_dict.filter_extremes(no_below = lower_bound,
                              no_above = upper_bound)
## Step 4: map words in dictionary to words in each
            document in the corpus
corpus_fromdict = [text_raw_dict.doc2bow(one_text)
                   for one_text in text_raw_tokens]
```

Run LDA

```
## Step 5: estimate a model by feeding it: (1) the corpus
## mapped to the dictionary, (2) the dictionary itself,
## (3) number of topics (in notation, k),
## and assorted other arguments
ldamodel = gensim.models.ldamodel.LdaModel(corpus_fromdict,
                                             num_topics = 5,
                                             id2word=text_raw_dict,
                                             passes=6,
                                             alpha = 'auto',
                                             per_word_topics = True)
```

Returns a trained Ldamodel class with various methods/attributes

n_topics is the most important parameter

Run Part 3